

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Катков Алексей Сергеевич

Группа К3339

Поток БР1.2

Проверил:

Добряков Давид Ильич

Санкт-Петербург

2026 г.

Задача

Реализовать REST API на основе выбранного boilerplate-проекта.

С учетом ранее спроектированной базы данных и OpenAPI-схемы необходимо реализовать модели, представления и контроллеры для выбранного варианта приложения. В результате выполнения лабораторной работы должно быть продемонстрировано полностью работающее API.

В качестве предметной области используется сервис обмена рецептами и кулинарных блогов.

Ход работы

Для выполнения лабораторной работы был использован boilerplate на основе Express, TypeORM и TypeScript. Данный шаблон уже содержал базовую структуру приложения, подключение к базе данных, JWT-авторизацию, поддержку контроллеров через **routing-controllers**, а также автоматическую генерацию Swagger-документации.

В качестве базы данных использовалась PostgreSQL. Для подключения к базе данных был настроен файл `.env`, в котором были указаны параметры подключения: хост, порт, имя базы данных, пользователь и пароль.

Основные параметры подключения:

- сервер приложения: `http://127.0.0.1:8000`;
- база данных: `recipe_api_lab1`;
- СУБД: PostgreSQL;
- ORM: TypeORM;
- документация API: Swagger UI по адресу `/docs`.

В ходе работы были реализованы основные сущности предметной области:

- **User** — пользователь сервиса;
- **Recipe** — рецепт, опубликованный пользователем;

- `Ingredient` — ингредиент рецепта;
- `DishType` — тип блюда;
- `RecipeStep` — шаг приготовления рецепта;
- `Comment` — комментарий пользователя к рецепту;
- `Like` — лайк рецепта;
- `SavedRecipe` — сохраненный рецепт пользователя;
- `Follow` — подписка одного пользователя на другого.

После создания моделей `TypeORM` автоматически создал соответствующие таблицы в базе данных. Между сущностями были настроены связи: пользователь может иметь несколько рецептов, рецепт может иметь несколько комментариев, шагов приготовления, ингредиентов и типов блюд. Также были предусмотрены связи для лайков, сохраненных рецептов и подписок.

Для работы с пользователями был доработан контроллер авторизации. В нем реализованы следующие методы:

- `POST /api/auth/register` — регистрация нового пользователя;
- `POST /api/auth/login` — авторизация пользователя.

При регистрации пользователь передает имя пользователя, email, пароль, имя и фамилию. Пароль сохраняется в базе данных в хэшированном виде. После успешной регистрации или входа API возвращает JWT-токен, который может использоваться для авторизации в защищенных методах.

Для работы с рецептами был реализован контроллер `RecipeController`. Он содержит основные CRUD-методы:

- `GET /api/recipes` — получение списка рецептов;
- `POST /api/recipes` — создание нового рецепта;
- `GET /api/recipes/details?id=` — получение рецепта по идентификатору;

- PATCH /api/recipes?id= — обновление рецепта;
- DELETE /api/recipes?id= — удаление рецепта.

При получении списка рецептов была реализована базовая фильтрация по названию и сложности рецепта. Это позволяет искать рецепты по части названия и отбирать их по уровню сложности: **easy**, **medium**, **hard**.

При создании рецепта пользователь передает название, описание, сложность, время приготовления, количество порций, ссылки на изображение и видео, а также идентификатор автора. После выполнения запроса рецепт сохраняется в базе данных.

Для социальной части приложения был реализован контроллер комментариев `CommentController`. Он содержит методы:

- GET /api/comments?recipeId= — получение комментариев конкретного рецепта;
- POST /api/comments — создание комментария.

При создании комментария передается текст комментария, идентификатор пользователя и идентификатор рецепта. API проверяет существование пользователя и рецепта, после чего сохраняет комментарий в базе данных.

Для проверки работы приложения использовался Swagger UI. Через Swagger были протестированы регистрация пользователя, авторизация, создание рецепта, получение списка рецептов, получение рецепта по идентификатору и создание комментария.

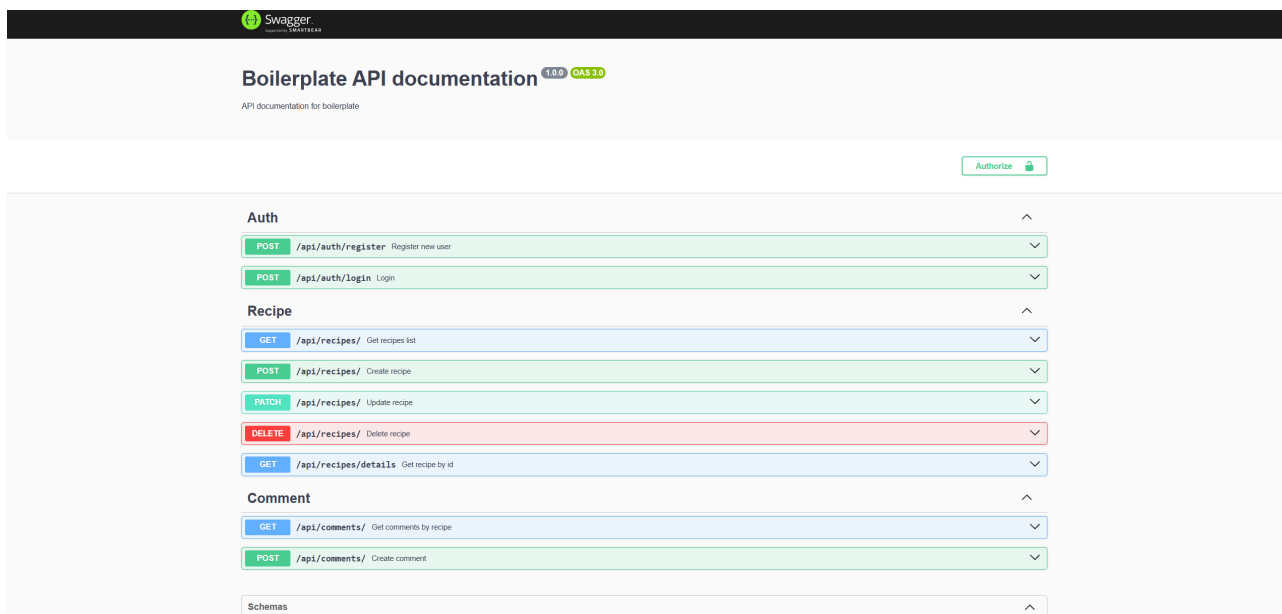


Рис. 1: Swagger-документация реализованного REST API

Также была проверена работа базы данных через PostgreSQL. После запуска приложения TypeORM создал таблицы для всех моделей, а после выполнения запросов в них появились тестовые данные.

Таким образом, в рамках лабораторной работы были реализованы модели, контроллеры и работающие REST-эндпоинты для сервиса обмена рецептами.

Вывод

В ходе лабораторной работы был реализован REST API для сервиса обмена рецептами и кулинарных блогов.

Для реализации использовался boilerplate на основе Express, TypeORM и TypeScript. Было настроено подключение к PostgreSQL, созданы модели предметной области, реализованы контроллеры авторизации, рецептов и комментариев.

Разработанное API позволяет регистрировать пользователей, выполнять авторизацию, создавать рецепты, получать список рецептов, получать рецепт по идентификатору, обновлять и удалять рецепты, а также добавлять и просматривать комментарии.

Работа API была проверена через Swagger UI. Полученная реализация соответствует выбранной предметной области и может быть расширена для

добавления лайков, сохраненных рецептов, подписок и более сложной фильтрации.